

# Smart Asset Inventory - App Overview

A cross-platform mobile app (Flutter/Dart, Android + iOS) for managing organizational assets end to end: inventory, custody transfers, maintenance work orders, procurement, risk, and AI failure prediction. It talks to two hosted backends: the AssetHub REST API (Vercel) for all business data, and a predictive-maintenance AI service (Railway) for per-asset failure-risk scores.

## 1. Architecture (lib/)

features/ (screens per domain: auth, dashboard, assets, custody, maintenance, procurement, locations, risk, settings) / core/services/ (one API client class per domain) / models/ (typed JSON contracts) / core/constants, core/theme, core/l10n, core/widgets (shared pieces). UI never touches HTTP directly; widgets call services, services return typed models.

## 2. Main features

- Auth: email login, access + refresh tokens, auto session-expiry logout.
- Role dashboards: Admin, Procurement, Custodian, Technician, Auditor - each sees its own cards, sections and actions; custodians are further limited to their location scope.
- Assets: search/filter, add/edit/delete, CSV import, detail with history.
- Custody & transfers: transfer requests; device outbox keeps refused requests as pending with retry.
- Maintenance: work-order lifecycle (open/assigned/in-progress/closed), due/overdue filters, templates, close-out with costs.
- Procurement: suppliers, purchase orders, invoices, warranties + expiring view.
- Risk & AI: local risk queue plus per-asset AI prediction (score, level, probability).
- Settings: light/dark theme + English/Arabic, persisted locally.

## 3. Key concepts used

- REST client (Dio): one base client with interceptors: auth header, 401 refresh-and-retry, request timing logs, global expired-session redirect.
- Token auth: access/refresh tokens in secure storage, in-memory user cache, logout clears everything.
- RBAC + scope: UI gates plus server-enforced permissions; location-subtree filtering for custodians.
- Async UI: FutureBuilder + setState for page data; one shared dashboard future; pull-to-refresh and Retry actions everywhere.
- Caching: short-TTL in-memory caches for locations, categories and user to avoid refetching static data.
- Pagination: page/limit (max 100); server filters first, client filters as backup.
- Defensive parsing: tolerant JSON models: bad records are skipped, wrapped shapes unwrapped, never a full-screen crash.
- Error UX: backend error envelope parsed into plain-language messages (500/404/403/timeout/offline); one automatic retry for cold-start flakes.
- Offline/dev: DEV MOCK flag for backend-free development; transfer outbox in SharedPreferences for permission-refused requests.
- i18n & theming: English/Arabic strings via app locale; light/dark ThemeMode through a global AppSettings notifier.
- Typed contracts: request/response models with fromJson/toJson; validation before sending (e.g. AI needs purchaseDate + lifetime).
- AI isolation: separate unauthenticated HTTP client so the main bearer token is never sent to the third-party AI host.
- Testing: unit tests for mappings/contracts plus widget tests; live-API test skips cleanly without internet.